འབྲུག་རྒྱལ་འཛིན་གཙུག་ལག་སློབ་
ཁྲི།

College of Science and Technology
Rinchending: Bhutan

WEB101
WEB APPLICATION FUNDAMEN
(SS2026)

Practical 3 Report

Submitted By;
Name: Ugyen Norbu
Student No : 02240369
Programme: BE 1SWE
Date: 15.04.2026

RUB Wheel of Academic Law: Academic Dishonesty

Section H2 of the Royal University of Bhutan's *Wheel of Academic Law* provides the following definition of academic dishonesty:

Academic dishonesty may be defined as any attempt by a student to gain an unfair advantage in any assessment. It may be demonstrated by one of the following:

1. **Collusion:** the representation of a piece of unauthorised group work as the work of a single candidate.
2. **Commissioning:** submitting an assignment done by another person as the student's own work.
3. **Duplication**: the inclusion in coursework of material identical or substantially similar to material which has already been submitted for any other assessment within the University.
4. **False declaration**: making a false declaration to receive special consideration by an Examination Board or to obtain extensions to deadlines or exemption from work.
5. **Falsification of data**: presentation of data in laboratory reports, projects, etc., based on work purported to have been carried out by the student, which has been invented, altered or copied by the student.
6. **Plagiarism**: the unacknowledged use of another's work as if it were one's own.

Examples are:

- verbatim copying of another's work without acknowledgement.
- paraphrasing of another's work by simply changing a few words or altering the order of presentation, without acknowledgement.
- ideas or intellectual data in any form presented as one's own without acknowledging the source(s).
- making significant use of unattributed digital images such as graphs, tables, photographs, etc., taken from test books, articles, films, plays, handouts, the internet, or any other source, whether published or unpublished.
- submission of a piece of work which has previously been assessed for a different award or module or at a different institution as if it were new work.
- use of any material without prior permission of copyright from the appropriate authority or owner of the materials used".

Table of Contents

SI No	TOPIC	Pages
1	ACKNOWLEDGEMENT	3
2	AIM AND OBJECTIVES	4
3	BACKGROUND INFORMATION	5
4	PART 1 – Setting Up the Project	6 - 7
5	PART 2 – Creating the API Route	8
6	PART 3 – Creating the Upload Form	9 - 14
7	CONCLUSION	15
8	REFERENCE	16

ACKNOWLEDGEMENT

First of all, I would like to thank the College of Science and Technology for giving me the important required resources for me to be able to do this practical. I am also very much thankful to our WEB module teacher, Mr. Prakash Chhetri, for helping us with his guidance and support while doing the assignment, where it helped me to understand better about the basics of web application development, not just rely on ai.

AIM AND OBJECTIVES

Aim

The main goal in this particular practical is to be able to create a simple file upload webpage by using Next.js where it enables the users to upload files into it by drag and droping it , I have also added a legit validation form while uploading.

Objectives

- Set the Next.js project using the correct configurations

- Install stuff like react-hook-form, axios and react-dropzone

- A valid upload form where it will receive inputs like name and files

- To allow only JPEG,PNG and PDF files, add validation

- Maintain file size limit at 5MB so that large files are rejected.

- Include drag and drop functions by react -dropzone

- Present preview after uploading the files

- Handle API routes properly to send and save the files

Background Information

These days, new web applications are made using different frameworks and tools to make the process faster, cleaner, and easier to handle over long durations, whereas in this particular assignment the big five techs used which are Next.js, React Hook Form, Axios, React Dropzone, and Tailwind CSS.

Next.js

The best web application building framework on top of React is the best for me, as it uses JavaScript to build the interactions or make it dynamic for the users. It enables us to be able to make smaller sections of the web page, which can be used again and again, called the components. Next.js makes extra functions on top of the react, like routing of files. This way, the folders inside the app folder are made into pages in the webpage, which makes moving between the pages is easy and simple.

React Hook Form

Is a very important dependence that makes handling legit forms with proper standard validation forms in the React applications. It gives easy functions to sign up, submit forms, and watch, that is making the inputs of the user to be to easir to collect with valid formats by setting rules when break of rule is broken, leading to pop up of error messages, such as when the inputs from the user is wrong. It was used in this assignment for the login and signup forms

Axios

Used for sending HTTP requests where it gets send to the backend API along with features like called onUploadProgress which will showcase the files uploaded.

React Dropzone

It offers the drag and drop function , limits the file size according to the rules which makes our life easir from instead of coding everything from scratch.

Tailwind CSS

The first class framework that makes styling of web pages inside the files using class names look like child's play, files such as HTML or JSX, rather than writing style codes for each line, use of Tailwind classes is able to be used directly to the components. This way the styling of the web application is much more faster and efficiently done for the whole web application.

These three main techs will allow us to create a good structure with attractive styling and workable Upload files functions or features in the web page.

Implementation/ PROCEDURE

Part 1 - Setting Up the Project

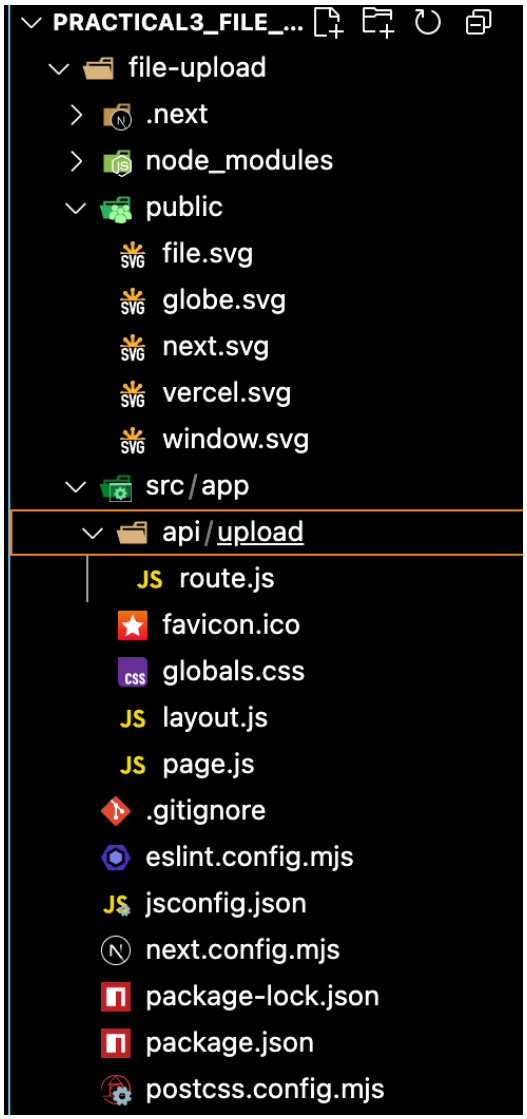
Step 1: Create a new Next.js project

```
deki@Ugays-MacBook file-upload % npx create-next-app file-upload
Need to install the following packages:
create-next-app@16.2.4
Ok to proceed? (y) y

✔ Would you like to use the recommended Next.js defaults? › No, customize settings
✔ Would you like to use TypeScript? … No / Yes
✔ Which linter would you like to use? › ESLint
✔ Would you like to use React Compiler? … No / Yes
✔ Would you like to use Tailwind CSS? … No / Yes
✔ Would you like your code inside a `src/` directory? … No / Yes
✔ Would you like to use App Router? (recommended) … No / Yes
✔ Would you like to customize the import alias (`@/*` by default)? … No / Yes
✔ Would you like to include AGENTS.md to guide coding agents to write up-to-date Next.js code? … No / Yes
Creating a new Next.js app in /Users/deki/Desktop/file-upload/file-upload.

Using npm.

Initializing project with template: default-tw


Installing dependencies:
- next
- react
- react-dom

Installing devDependencies:
- @tailwindcss/postcss
- eslint
- eslint-config-next
- tailwindcss


added 354 packages, and audited 355 packages in 2m

143 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities


 * History restored
```

Step 2: Install required dependencies

```
deki@Ugays-MacBook file-upload % npm install react-hook-form axios react-dropzone formidable

changed 3 packages, and audited 66 packages in 5s

15 packages are looking for funding
  run `npm fund` for details

1 moderate severity vulnerability

To address all issues, run:
  npm audit fix

Run `npm audit` for details.
```

Step 3: Clean up and set up the folder structure



Part 2 - Creating the API Route

Step 1: Create the API route file at **src/app/api/upload/route.js**

JS route.js

file-upload > src > app > api > upload > JS route.js > POST

```javascript
// src/app/api/upload/route.js
import { writeFile, mkdir } from 'fs/promises';
import { NextResponse } from 'next/server';
import { join } from 'path';
import path from 'path';

Tabnine | Edit | Test | Explain | Document
export async function POST(request) {
  try {
EB/SS2026_WEB101_02240369/Practical3_file_upload/file-
/next.svg
    if (!file) {
      return NextResponse.json(
        { error: 'No file uploaded' },
        { status: 400 }
      );
    }
    const bytes = await file.arrayBuffer();
    const buffer = Buffer.from(bytes);
    // Create the uploads directory if it doesn't exist
    const uploadDir = join(process.cwd(), 'public', 'uploads');
    try {
      await mkdir(uploadDir, { recursive: true });
    } catch (error) {
      if (error.code !== 'EEXIST') {
        throw error;
      }
    }
    // Create a unique filename using timestamp
    const uniqueFilename = `${Date.now()}-${file.name}`;
    const filePath = join(uploadDir, uniqueFilename);
    await writeFile(filePath, buffer);
    return NextResponse.json({
      message: 'File uploaded successfully',
      filename: uniqueFilename,
      originalName: file.name,
      url: `/uploads/${uniqueFilename}`
    });
  } catch (error) {
    console.error('Error uploading file:', error);
    return NextResponse.json(
      { error: `Error uploading file: ${error.message}` },
      { status: 500 }
    );
```

Part 3 - Creating the Upload Form

Step 1: Create the file upload form in src/app/page.js

file-upload > src > app > JS page.js > FileUploadForm

```javascript
"use client"
import { useState, useCallback } from 'react';
import { useForm, Controller } from 'react-hook-form';
import axios from 'axios';
import { useDropzone } from 'react-dropzone';
Tabnine | Edit | Test | Fix | Explain | Document
export default function FileUploadForm() {
  // Validation constants
  const MAX_FILE_SIZE = 5 * 1024 * 1024; // 5MB
  const ACCEPTED_FILE_TYPES = ['image/jpeg', 'image/png', 'application/pdf'];
  // React Hook Form setup
  const { control, register, handleSubmit, formState: { errors } } = useForm();
  // State for upload progress and results
  const [uploadProgress, setUploadProgress] = useState(0);
  const [isUploading, setIsUploading] = useState(false);
  const [uploadResult, setUploadResult] = useState(null);
  const [filePreview, setFilePreview] = useState(null);
  // Form submit handler
```

```javascript
  // Form submit handler
  const onSubmit = async (data) => {
    setIsUploading(true);
    setUploadProgress(0);
    setUploadResult(null);
    try {
      const formData = new FormData();
      formData.append('file', data.file[0]);
      formData.append('name', data.name);
      console.log('Uploading file:', data.file[0].name, 'Type:', data.file[0].type);
      const response = await axios.post('http://localhost:8000/api/upload', formData, {
        onUploadProgress: (progressEvent) => {
          const percentage = Math.round(
            (progressEvent.loaded * 100) / progressEvent.total
          );
          setUploadProgress(percentage);
        }
      });
      setUploadResult({
        success: true,
        message: 'File uploaded successfully!',
        data: response.data
      });
    } catch (error) {
      console.error('Upload error:', error);
      setUploadResult({
        success: false,
        message: error.response?.data?.error || 'Upload failed'
      });
    } finally {
      setIsUploading(false);
    }
  };
```

```
// Drag & Drop component
const Dropzone = ({ onDrop, maxSize, accept }) => {
  const {
    getRootProps,
    getInputProps,
    isDragActive,
    fileRejections
  } = useDropzone({
    onDrop: (acceptedFiles) => {
      onDrop(acceptedFiles);
      if (acceptedFiles.length > 0) {
        const file = acceptedFiles[0];
        if (file.type.startsWith('image/')) {
          const previewUrl = URL.createObjectURL(file);
          setFilePreview({ url: previewUrl, name: file.name, type: file.type });
        } else if (file.type === 'application/pdf') {
          setFilePreview({ name: file.name, type: file.type });
        } else {
          setFilePreview(null);
        }
      }
    },
    maxSize,
    accept,
    multiple: false
  });
  const fileRejectionItems = fileRejections.map(({ file, errors }) => (
    <li key={file.path} className="text-red-500">
      {file.name} - {errors.map(e => e.message).join(', ')}
    </li>
  ));
  return (
    <div className="mt-1">
      <div
        {...getRootProps()}
        className={`border-2 border-dashed rounded-lg p-6 cursor-pointer text-center ${
          isDragActive ? 'border-blue-500 bg-blue-50' : 'border-gray-300 hover:border-blue-400'
        }`}
      >
        <input {...getInputProps()} />
        {isDragActive ? (
          <p className="text-blue-500">Drop the file here...</p>
        ) : (
          <div>
            <p>Drag & drop a file here, or click to select</p>
            <p className="text-sm text-gray-500 mt-1">
              (Only JPEG, PNG, and PDF files under 5MB are accepted)
            </p>
          </div>
        )}
      </div>
      {fileRejectionItems.length > 0 && (
        <ul className="mt-2">{fileRejectionItems}</ul>
      )}
    </div>
  );
};
return (
  <div className="min-h-screen flex flex-col items-center justify-center p-4">
    <h1 className="text-2xl font-bold mb-6">File Upload Form</h1>
    <form onSubmit={handleSubmit(onSubmit)} className="max-w-md">
```

```
111        {/* Name Field */}
112        <div className="mb-4">
113          <label htmlFor="name" className="block mb-1 font-medium">Your Name:</label>
114          <input
115            id="name"
116            type="text"
117            {...register('name', { required: 'Name is required' })}
118            className="w-full p-2 border rounded"
119          />
120          {errors.name && <p className="text-red-500 mt-1">{errors.name.message}</p>}
121        </div>
122        {/* File Upload Field */}
123        <div className="mb-4">
124          <label className="block mb-1 font-medium">Upload File:</label>
125          <Controller
126            name="file"
127            control={control}
128            rules={{
129              required: 'File is required',
130              validate: {
131                fileSize: files => !files?.[0] || files[0].size <= MAX_FILE_SIZE ||
132                  'File size must be less than 5MB',
133                fileType: files => !files?.[0] || ACCEPTED_FILE_TYPES.includes(files[0].type) ||
134                  'Only JPEG, PNG, and PDF files are accepted'
135              }
136            }}
137            render={({ field: { onChange } }) => (
138              <Dropzone
139                onDrop={acceptedFiles => onChange(acceptedFiles)}
140                maxSize={MAX_FILE_SIZE}
141                accept={{
142                  'image/jpeg': ['.jpg', '.jpeg'],
143                  'image/png': ['.png'],
144                  'application/pdf': ['.pdf']
145                }}
146              />
147            )}
148          />
149          {errors.file && <p className="text-red-500 mt-1">{errors.file.message}</p>}
150        </div>
151        {/* File Preview */}
152        {filePreview && (
153          <div className="mb-4">
154            <h3 className="font-medium mb-1">Preview:</h3>
155            <div className="border rounded p-2">
156              {filePreview.type?.startsWith('image/') ? (
157                // eslint-disable-next-line @next/next/no-img-element
158                <img
159                  src={filePreview.url}
160                  alt={filePreview.name}
161                  className="max-w-full h-auto max-h-40 rounded"
162                />
163              ) : filePreview.type === 'application/pdf' ? (
164                <div className="py-2 px-3 bg-gray-100 rounded flex items-center">
165                  <svg className="w-6 h-6 text-red-500 mr-2" fill="currentColor" viewBox="0 0 20 20">
166                    <path d="M9 2a2 2 0 00-2 2v8a2 2 0 002 2h6a2 2 0 002-2V6.414A2 2 0 0016.414 5L14 2.586A2 2 0 0012.586 2H9z"></p
167                    <path d="M3 8a2 2 0 012-2h2a2 2 0 012 2v8a2 2 0 01-2 2H5a2 2 0 01-2-2V8z"></path>
168                  </svg>
169                  <span>{filePreview.name}</span>
170                </div>
171              ) : (
172                <div>File selected: {filePreview.name}</div>
173              )}
174            </div>
175          </div>
176        )}
```

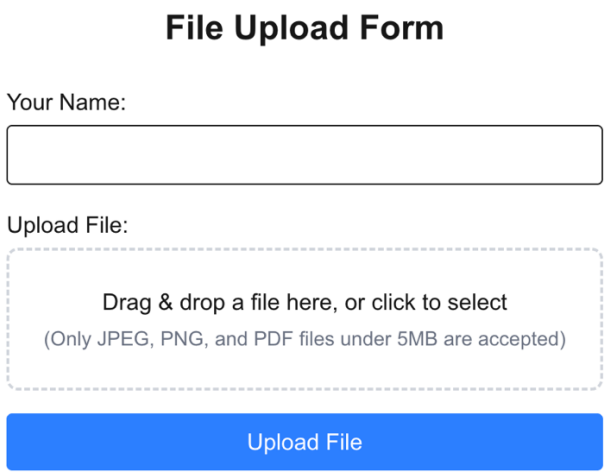
```
177          {/* Submit Button */}
178          <button
179            type="submit"
180            disabled={isUploading}
181            className={`w-full p-2 text-white rounded ${
182              isUploading ? 'bg-blue-400' : 'bg-blue-500 hover:bg-blue-600'
183            }`}
184          >
185            {isUploading ? 'Uploading...' : 'Upload File'}
186          </button>
187          {/* Progress Bar */}
188          {isUploading && (
189            <div className="mt-4">
190              <div className="w-full bg-gray-200 rounded-full h-2.5">
191                <div
192                  className="bg-blue-600 h-2.5 rounded-full transition-all duration-300"
193                  style={{ width: `${uploadProgress}%` }}
194                ></div>
195              </div>
196              <p className="text-center mt-1 text-sm">{uploadProgress}%</p>
197            </div>
198          )}
199          {/* Result Message */}
200          {uploadResult && (
201            <div className={`mt-4 p-3 rounded ${
202              uploadResult.success ? 'bg-green-100 text-green-800' : 'bg-red-100 text-red-800'
203            }`}>
204              <p>{uploadResult.message}</p>
205              {uploadResult.success && uploadResult.data?.filename && (
206                <p className="text-sm mt-1">Uploaded as: {uploadResult.data.filename}</p>
207              )}
208            </div>
209          )}
210        </form>
211      </div>
212    );
213  }
```

Step 2: Start the development server

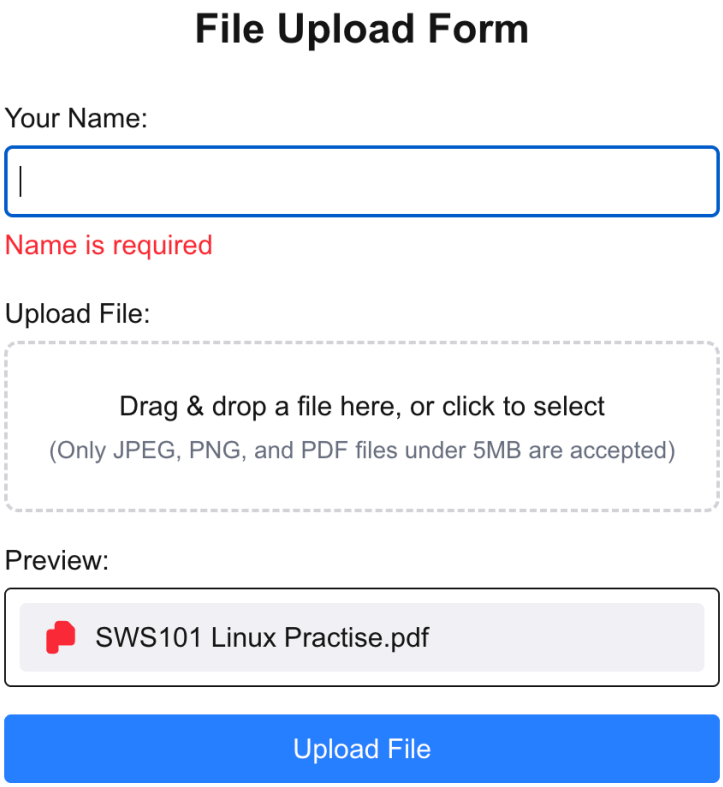
```
● deki@Ugays-MacBook Practical3_file_upload % cd file-upload
○ deki@Ugays-MacBook file-upload % npm run dev

  > file-upload@0.1.0 dev
  > next dev

  ⚠ Warning: Next.js inferred your workspace root, but it may not be correct.
   We detected multiple lockfiles and selected the directory of /Users/deki/package-lock.json as the root directory.
   To silence this warning, set `turbopack.root` in your Next.js config, or consider removing one of the lockfiles if it's not needed.
     See https://nextjs.org/docs/app/api-reference/config/next-config-js/turbopack#root-directory for more information.
   Detected additional lockfiles:
     * /Users/deki/Desktop/WEB/SS2026_WEB101_02240369/Practical3_file_upload/file-upload/package-lock.json

  ▲ Next.js 16.1.7 (Turbopack)
  - Local:         http://localhost:3000
  - Network:       http://10.2.32.181:3000

  ✓ Starting...
  ✓ Ready in 717ms
```

Part 4 - Testing the Application

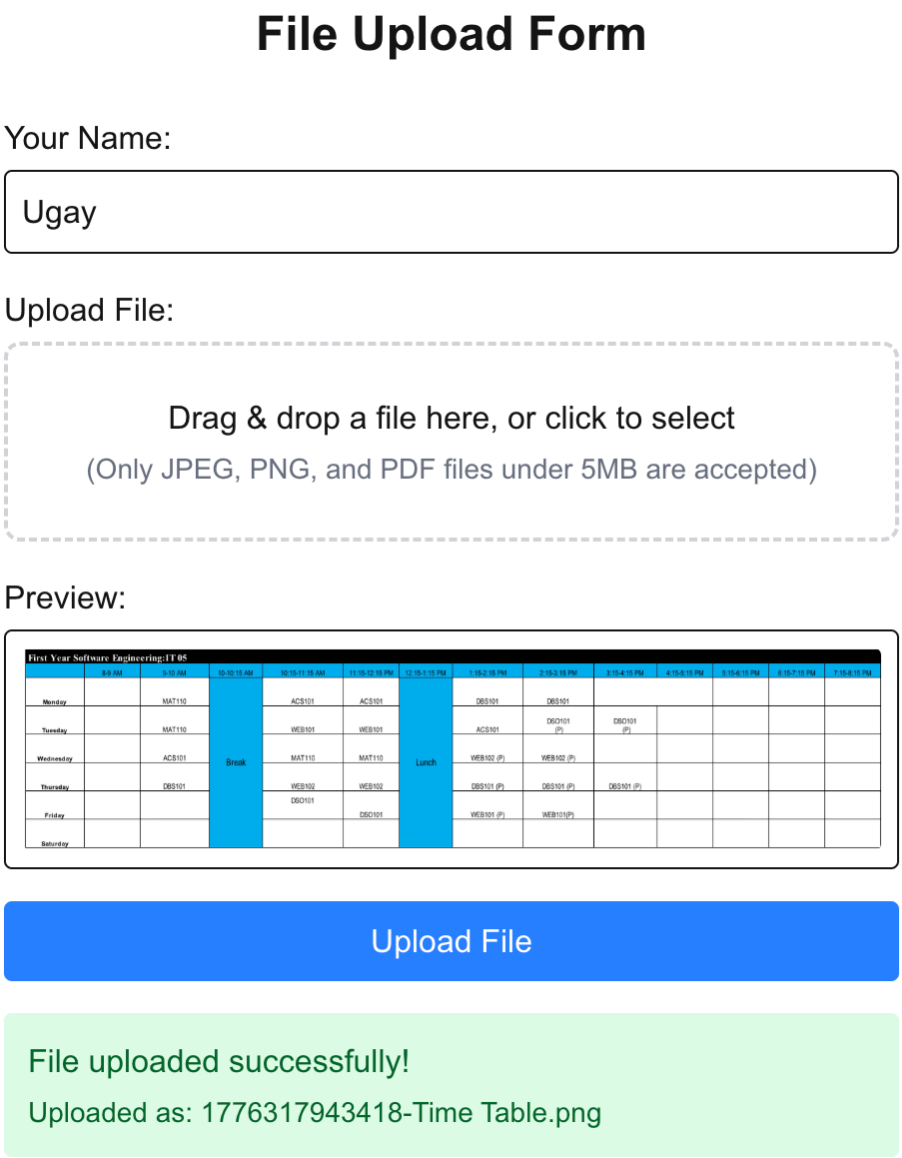
Step 1: Open the app in the browser at **http://localhost:3000**



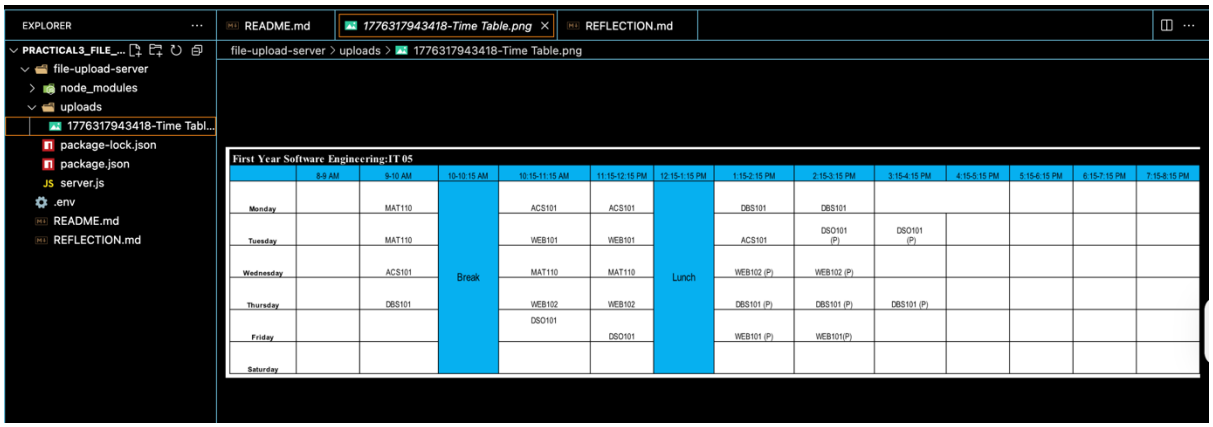
Step 2: Test form validation ,try submitting without a name



Step 4: Upload a valid file and check the success message



Step 5: Check that the file was saved



CONCLUSION

Finally, after a lot of errors and syntax I was finally able to successfully make a workable File Upload web application, even if it is just locally by using frameworks like Next.js, Tailwind CSS and library React Hook Form. This project is divided in 3 major parts. Part 1, where I set up the Next.js project folders, styling using the Tailwind CSS and make basic folder structure with all the necessary dependences. Part 2, I made proper API routes or paths that will save the files in the server or backend. In part 3, I focused on making the upload form along with drag and drop functions, proper validation, and file preview section.

I faced a lot of problems or issues like apostrophe syntax in the JSX files, I had many issues where I was confused about the app router folder and where to put the API routes since all of this was my first time working as a website developers that to only frontend.The ESLint gave a lot of warning on the image tags which made me think it was broken but later found out that is was only suggestions not errors , also the centering the from was kind of plain so had to use Tailwind. However after all the errors, I was getting used to it over time with more practice and help from my tutors where I had to carefully use my brains to analyse each error messeges and make the required edits or corrections.

This assignment has given me the chance to learn how file upload actually works in full stack apps such as files uploaded in fronend gets saved in backend using APIs which is my first time dealing with these devices where I found the best practice method with React Hook Form and Axios was the best to arrange or organize and reuse of codes. This new knowledge will definitely be useful down the line in my career path or soon to come assignments, and I will be looking forward to it in the future.

REFERENCES

Vercel. (n.d.). *Getting started with Next.js*. Next.js Documentation. https://nextjs.org/docs

React Hook Form. (n.d.). *Getting started*. React Hook Form Documentation. https://react-hook-form.com/get-started

React Dropzone. (n.d.). *Getting started*. React Dropzone Documentation. https://react-dropzone.js.org

Axios. (n.d.). *Getting started*. Axios Documentation. https://axios-http.com/docs/intro

Tailwind Labs. (n.d.). *Installation*. Tailwind CSS Documentation. https://tailwindcss.com/docs/installation